

Documento de Arquitectura

Arquitectura General del Sistema

La solución implementada corresponde a un asistente conversacional desarrollado sobre Amazon Bedrock utilizando Claude Sonnet como motor de razonamiento. El sistema permite consultar información almacenada tanto en Amazon Athena como en Amazon S3 mediante lenguaje natural.

La arquitectura se divide en dos grandes componentes:

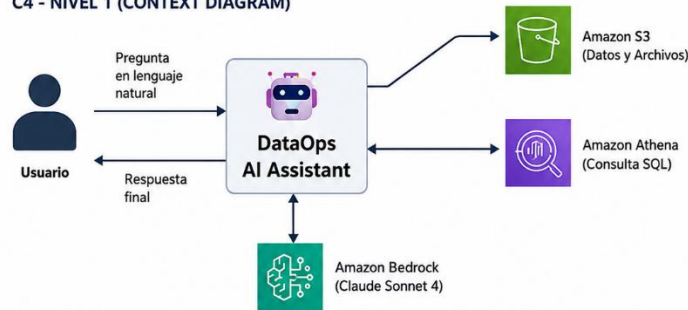
- Frontend desarrollado en React.
- Backend desarrollado en FastAPI.

El Backend incorpora un agente conversacional encargado de interpretar la intención del usuario, decidir si requiere utilizar una herramienta externa (Tool) y posteriormente generar una respuesta utilizando Claude Sonnet.

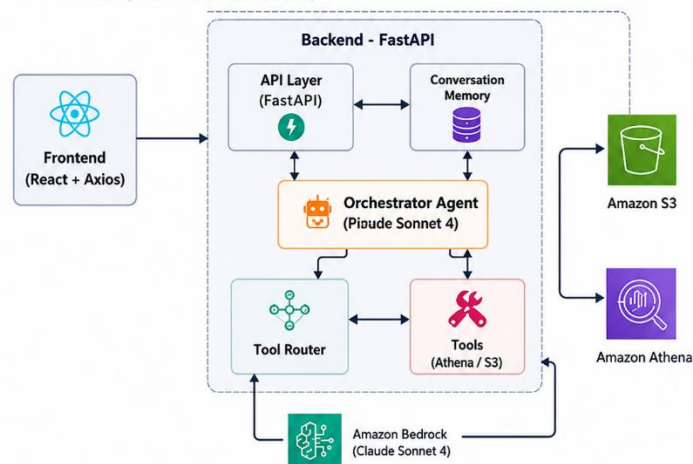
Los datos nunca son consultados directamente por el modelo LLM, sino mediante herramientas especializadas que consultan los servicios AWS.

2. ARQUITECTURA DE ALTO NIVEL

C4 - NIVEL 1 (CONTEXT DIAGRAM)



C4 - NIVEL 2 (CONTAINER DIAGRAM)



Arquitectura C4 - Nivel 1

Los componentes principales son:

Usuario

Realiza preguntas utilizando lenguaje natural desde el navegador web.



Frontend (React)

Interfaz gráfica donde el usuario interactúa con el asistente.



Backend (FastAPI)

Expone la API REST /api/v1/chat.



Agente Conversacional

Interpreta la intención del usuario.



Amazon Bedrock (Claude Sonnet 4)

Genera el razonamiento y la respuesta.



Herramientas

- Athena Tool
- S3 Tool



Servicios AWS

- Amazon Athena
- Amazon S3

Arquitectura C4 - Nivel 2

Componentes internos del Backend

API Layer

Responsabilidad

Recibir solicitudes HTTP provenientes del Frontend.

Entrada

Mensaje del usuario.

Salida

Respuesta generada por el agente.

DataOps Agent

Responsabilidad

Orquestar todo el proceso conversacional.

Funciones

- Mantener historial
- Detectar herramientas
- Ejecutar herramientas
- Construir prompts
- Invocar Claude Sonnet
- Devolver respuesta

Tool Router

Responsabilidad

Analizar la intención del usuario para decidir qué herramienta utilizar.

Actualmente soporta:

- Athena Tool
- S3 Tool

Conversation Memory

Responsabilidad

Mantener el historial de la conversación para preservar contexto entre preguntas.

Bedrock Repository

Responsabilidad

Enviar el prompt a Claude Sonnet mediante Amazon Bedrock Runtime.

Agentes Implementados

Agente Orquestador

Responsabilidades

- Recibir la solicitud del usuario.
- Consultar el historial.
- Detectar si la pregunta requiere herramientas.
- Ejecutar la herramienta correspondiente.
- Construir el prompt final.
- Invocar Claude Sonnet.
- Retornar la respuesta.

Entradas

- Pregunta del usuario.
- Historial de conversación.

Salidas

- Respuesta final en Markdown.

Sub-workers

En la implementación desarrollada no se construyeron agentes secundarios autónomos.

En su lugar, la arquitectura implementa un conjunto de herramientas (Tools) especializadas que cumplen una función equivalente.

Athena Tool

Responsabilidad:

Consultar Amazon Athena.

Entrada

Consulta SQL.

Salida

Resultados estructurados.

S3 Tool

Responsabilidad:

Consultar Amazon S3.

Entrada

Ruta o bucket.

Salida

Listado de archivos.

Agente Juez

No fue implementado en esta primera versión del proyecto.

La validación de respuestas depende actualmente de Claude Sonnet y de la evidencia devuelta por las herramientas.

Como mejora futura se propone incorporar un Agente Juez que evalúe:

- consistencia
- precisión
- alucinaciones
- formato de respuesta

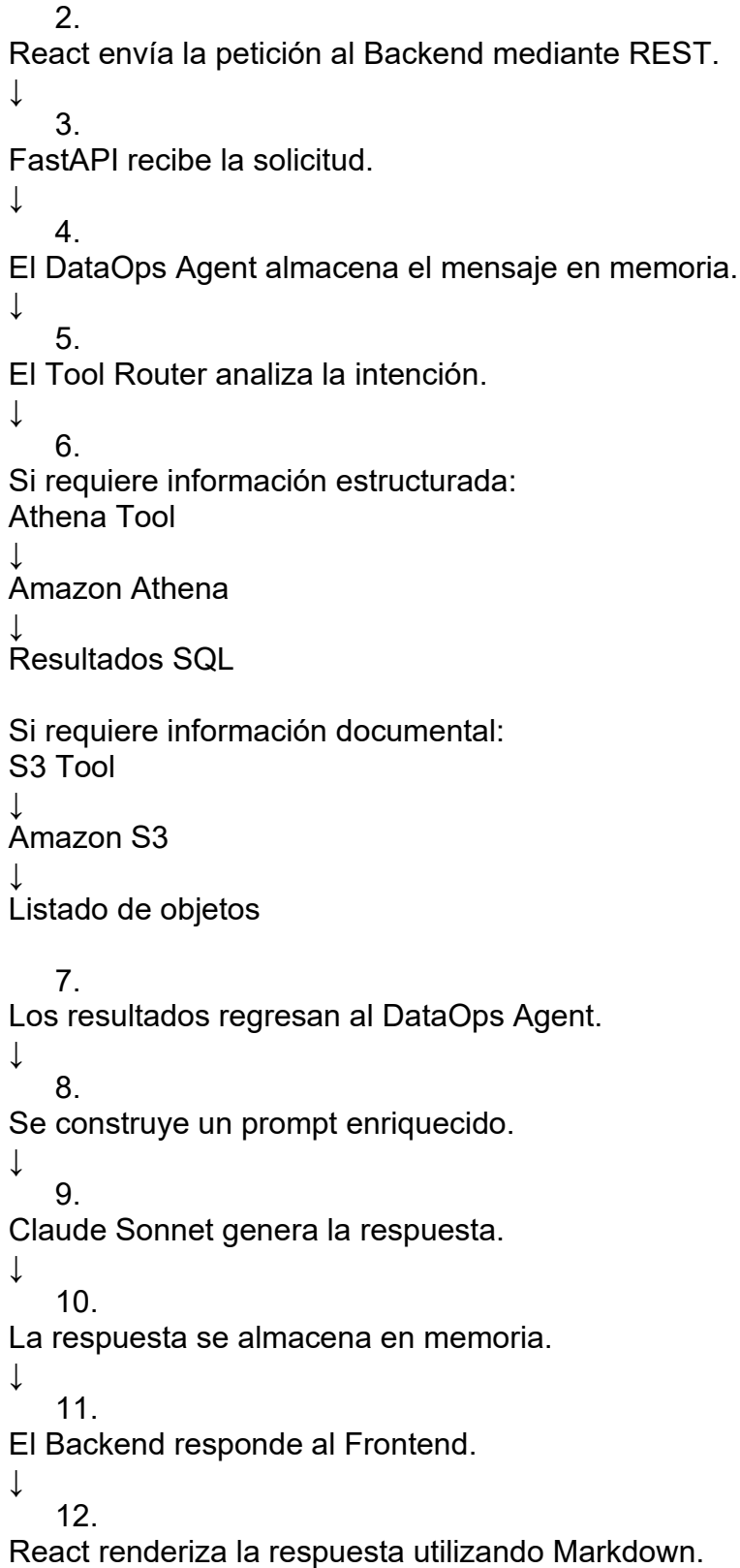
antes de responder al usuario.

Flujo de Datos

1.

El usuario escribe una pregunta.

↓



Selección del LLM

Comparativa

MODELO	VENTAJAS	DESVENTAJAS
CLAUDE SONNET 4	Excelente razonamiento, muy buena generación de SQL, baja alucinación, integración nativa con Bedrock	Precio superior a Haiku
CLAUDE HAIKU 3.5	Muy rápido y económico	Menor capacidad de razonamiento complejo
GPT-4.1	Muy potente, excelente generación de código	Integración fuera del ecosistema AWS
LLAMA 3.1	Open Source	Requiere infraestructura propia

¿Por qué Claude Sonnet?

Se seleccionó Claude Sonnet debido a que la solución debía ejecutarse completamente sobre AWS utilizando Amazon Bedrock.
Las principales razones fueron:

Excelente razonamiento

Las consultas realizadas por el usuario requieren interpretar lenguaje natural y transformarlo en acciones concretas sobre herramientas AWS.
Claude Sonnet presenta uno de los mejores desempeños actuales en tareas de razonamiento.

Excelente generación de SQL

Gran parte de la solución consulta Amazon Athena.
Claude Sonnet genera consultas SQL mucho más consistentes que otros modelos pequeños.

Baja tasa de alucinaciones

Al trabajar con herramientas externas (Tool Calling) resulta fundamental minimizar respuestas inventadas.
Claude Sonnet presenta un comportamiento mucho más conservador que otros modelos.

Integración nativa con Amazon Bedrock

No fue necesario desplegar infraestructura propia.
Toda la inferencia se realiza mediante Bedrock Runtime.
Esto mejora:

- seguridad
- escalabilidad
- administración

Seguridad

Los datos nunca abandonan el entorno AWS.

Toda la comunicación ocurre mediante IAM y Bedrock Runtime.

Relación costo-beneficio

Aunque Claude Sonnet es más costoso que Haiku, ofrece una mayor precisión en consultas empresariales, reduciendo errores y respuestas incorrectas.

Para un asistente DataOps resulta más importante la precisión que una diferencia marginal en costo por consulta.

Estrategia de Chunking

En la versión implementada no fue necesario realizar chunking.

La solución consulta información estructurada desde:

- Amazon Athena
- Amazon S3

No se implementó un motor RAG ni procesamiento de documentos extensos.

Como mejora futura se propone utilizar:

Tamaño del chunk

1000 tokens

Overlap

200 tokens

Separadores

- párrafos
- encabezados
- tablas

Schema JSON de las Tools

- **Athena Tool**

```
{
  "name": "athena",
  "description": "Ejecuta consultas SQL sobre Amazon Athena.",
  "parameters": {
    "query": {
      "type": "string",
      "required": true
    }
  }
}
```

```

    }
  },
  "returns": {
    "type": "array"
  }
}

```

- **S3 Tool**

```

{
  "name": "s3",
  "description": "Consulta archivos almacenados en Amazon S3.",
  "parameters": {
    "query": {
      "type": "string",
      "required": false
    }
  },
  "returns": {
    "type": "array"
  }
}

```

Diagrama de Despliegue AWS

Servicios utilizados:

- Amazon Bedrock Runtime
- Claude Sonnet
- Amazon Athena
- Amazon S3
- AWS IAM
- AWS Glue Data Catalog
- FastAPI
- React

Estimación de Costos

Supuestos:

- 500 consultas por día.
- 15.000 consultas mensuales.
- 2.000 tokens de entrada.
- 1.000 tokens de salida.

Servicio	Costo aproximado mensual
Amazon Bedrock (Claude Sonnet)	USD 220–320
Amazon Athena	USD 5–10
Amazon S3	< USD 2
AWS Glue Catalog	Sin costo relevante
Transferencia	< USD 5

Costo mensual estimado: USD 230–340, dependiendo del número real de tokens procesados y del volumen de datos escaneados por Athena.

Decisiones de Diseño

FastAPI

Seleccionado por:

- alto rendimiento
- documentación automática (Swagger)
- integración sencilla con Python

React

Seleccionado por:

- interfaz moderna
- componentes reutilizables
- integración sencilla con APIs REST

Amazon Athena

Seleccionado porque permite consultar archivos CSV directamente desde Amazon S3 sin necesidad de crear una base de datos relacional ni procesos ETL adicionales.

Amazon S3

Utilizado como Data Lake para almacenar archivos CSV en la capa Bronze, aprovechando su alta durabilidad, bajo costo y escalabilidad prácticamente ilimitada.

Amazon Bedrock

Permite consumir modelos fundacionales como Claude Sonnet sin administrar infraestructura de inferencia, ofreciendo integración nativa con IAM, seguridad empresarial y escalabilidad administrada.

Claude Sonnet

Seleccionado por su excelente capacidad de razonamiento, generación de SQL, baja tasa de alucinaciones y desempeño superior para escenarios de asistentes empresariales que utilizan herramientas externas.

Arquitectura basada en Tools

Se optó por una arquitectura con herramientas especializadas (Athena Tool y S3 Tool) en lugar de permitir que el modelo responda únicamente con su conocimiento. Esto garantiza que las respuestas se fundamenten en datos reales obtenidos desde AWS, reduciendo significativamente el riesgo de alucinaciones y mejorando la confiabilidad del sistema.